

C++ 课程大作业（一）：高性能计算挑战——极限阶乘

对象：自愿报名，个人独立完成 | 周期：第 2~5 周 | 截止：第 5 周周日 24:00

1. 项目背景

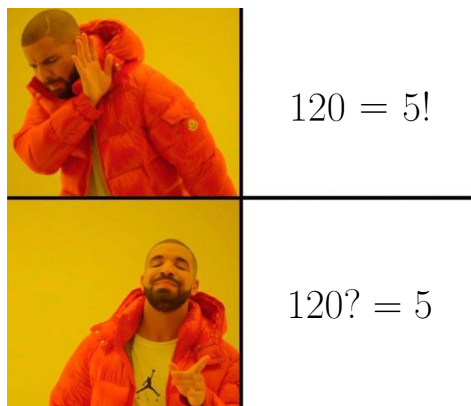


图 1: 我对阶乘逆符号的提议: r/mathmemes

阶乘计算看似简单，但当 n 达到 10^8 量级时，它就不再是一个数学问题，而是一个涉及算法优化、数据结构设计与工程实现的综合挑战。

$$\log_{10}(10^8!) \approx 7.6 \times 10^8$$

即约 **7.6** 亿位十进制数字。

2. 命题任务

■ 在给定的峰值内存预算内，算出 $n!$ 的精确值。

2.1 你要算的 n

为保证每个人的任务互不相同， n 由你的学号决定。

设 x 为你学号的末两位，对难度系数 $k \in \{4, 5, 6, 7, 8\}$ ，你要计算的 n 为：

$$n = 10^k + x \times 10^3$$

从 $k = 4$ 开始逐档往上，能过到哪一档就算哪一档。

示例：若你的学号末两位是 $x = 7$ ，

- $k = 4$: 17,000! (入门级)
- $k = 5$: 107,000! (进阶级)
- $k = 6$: 1,007,000! (挑战级)
- $k = 7$: 10,007,000! (专家级)
- $k = 8$: 100,007,000! (魔鬼级)

2.2 资源预算

项目	限制
峰值内存	3 GB （全档统一，且禁用 swap）
单档时限	600 s
线程数	由评测脚本统一固定为 8 ，不由你的程序决定

超内存、超时、或输出指纹不符，该档即判未通过。

2.3 禁用与允许

禁止使用任何第三方大整数 / 数论 / FFT 库。点名禁止（不限于）：GMP、MPIR、MPFR、FLINT、NTL、Boost.Multiprecision，以及任何提供大整数乘法、FFT/NTT 或模运算的现成实现。

明确允许：

- C++ 标准库（`<vector>`、`<complex>`、`<cstdint>` 等）；
- 编译器扩展 `__int128` / `unsigned __int128`——它正是高难度档的通关钥匙；
- OpenMP、`std::thread`、SIMD intrinsics；
- 任意编译选项，如 `-O3 -march=native`。

判断标准：你的程序能否只靠 `g++` 和标准库编出来。不确定某样东西算不算库，动手前问助教，不要事后再说。

2.4 附加挑战（表演档）

有余力可以挑战 $3 \times 10^8!$ ：不限内存、在你自己的机器上运行、不进主排行榜，但可以在讲评课上展示，需报告机器配置。现有两条参考线：

实现	路线	用时	峰值内存
去年最优学生实现（8 线程）	手写 NTT、 10^{14} 进制、双模数 CRT、Montgomery、平衡乘积树	210 s	8.0 GiB
AI 零介入实现（8 核笔记本）	调库，乘法交给 FLINT 的多模数 FFT	294 s	10.9 GB

注意：此量级结果极大，对内存和磁盘 I/O 均有极高要求，请谨慎尝试。

3. 程序接口与评测协议

3.1 程序接口

你的程序从命令行读入 n ，向 `stdout` 输出恰好 **5** 行，格式如下（这里用 $1000!$ 举例，五个数都是真的，可以拿来自测）：

```
$ ./solution 1000
DIGITS    2568
ZEROS     249
DIGITSUM  10539
HEAD50    40238726007709377354370243392300398571937486421071
TAIL50    43416909004153690105933983835777939410970027753472
```

五项分别是：DIGITS = 结果的总位数（十进制长度）；ZEROS = 末尾 0 的个数；DIGITSUM = 所有数位之和；HEAD50 = 结果的前 50 位；TAIL50 = 去掉末尾所有 0 之后的最后 50 位。

（TAIL50 之所以要先去掉末尾的 0： $10^8!$ 的末尾有两千五百万个 0，直接取末 50 位只会得到 50 个 0，不含任何信息。）

3.2 评测环境

统一在助教的评测机上运行，跑在容器里：Intel i7-10700（8 核 16 线程）、16 GB 双通道 DDR4-2667、Ubuntu 24.04 容器、g++ 13.3、-std=c++17。

```
docker run --rm --memory=3g --memory-swap=3g \
  --cpuset-cpus=0-7 --network=none \
  -e OMP_NUM_THREADS=8 -v "$PWD:/work:ro" -w /work \
  judge-cpp:13 timeout 600 ./solution <n>
```

--memory-swap 与 --memory 取同值，等价于把 swap 额度设为 0：超预算就地被杀，不能靠换页硬撑。
--cpuset-cpus=0-7 绑定的是 8 个不同的物理核，超线程孪生核不参与。

注意一个坑：容器里 std::thread::hardware_concurrency() 返回的是宿主机的 16，不是你实际能用的 8。用 std::thread 请读环境变量 OMP_NUM_THREADS，否则你会在 8 个核上开 16 个线程。

本地自测请采用同样的方法（3 次取最小、关掉其他程序），否则你的数字和评测机的数字没有可比性。

最后提示一处硬件特性：NTT 蝶形变换是访存带宽受限的操作，8 核并行远拿不到 8 倍提速。因此靠前的名次可能部分取决于你的访存模式是否友好，而不只是并行写得好不好。

4. 推荐技术路线

这是一个开放性问题，以下是建议的台阶，不是必须遵循的路径。

基础路线

- 分治乘积树：不要直接 for (i=1; i<=n; ++i)。用 solve(l, r) 先算左半部分的积，再乘右半部分的积——目的是让两个乘数的规模接近，而不是反复做「大数乘小数」。
- 数论优化：用 Legendre 定理把 $n!$ 拆成质因子的幂，或用 PrimeSwing，减少大整数乘法的频次与规模。
- FFT/NTT 加速乘法：位数超过一定阈值后， $O(N^2)$ 的竖式乘法不可用；而 FFT 的浮点精度在本题的规模下会不够，必须转向 NTT（快速数论变换），做到 $O(N \log N)$ 。

你会撞上的几道坎

- 默认模数的变换长度限制： $998244353 = 119 \times 2^{23} + 1$ ，只支持 $2^{23} \approx 8.4 \times 10^6$ 的变换长度。高难度档压位后的 limb 数远超这个值。绕开的办法是自选大质数模（如 $197 \times 2^{55} + 1$ ）并用 unsigned __int128 承载，或者多模数分段。
- 压位与基数选择： 10^{14} 进制是常见选择。基数选小了，变换长度会更早撞上上面那道坎。
- 多模数 CRT：保证 NTT 的系数不溢出。
- 二进制 → 十进制转换：实测中最耗时的一步，且最常被忽略——你很可能发现它比阶乘本身还慢。
- 常数优化：Montgomery 约减、SIMD、cache 友好的访存顺序。

5. 排名与评分

主赛道：按通关的最高 k 分档，同档内按该档的运行时间排名。

排行榜上另标一条 **AI** 零介入基线：由助教事前在无人工干预条件下跑出并公布位置。你的成绩高出基线多少，就是你自己贡献的可见度量。

评分：

1. 正确性与名次（**50%**）：必须通过 $k = 4$ 档；通关的 k 越高、用时越短，得分越高。
2. 可复现与易用性（**25%**）：编译与运行流程清晰、无需手工干预；校验数据齐全；你能说清楚凭什么相信自己算对了。
3. 创新性与 **AI** 使用（**15%**）：是否采用了本文档之外的合理优化策略，这些策略是否有效；是否高效地把 **AI** 用作老师而不是代笔。
4. 讲评展示（**10%**）：在第 6 周的讲评课上讲解自己的实现——思路、踩过的坑、为什么这么选。

第 4 项需自愿报名。不报名不扣分，但这 10 分自动放弃，总分上限为 90。报名不看名次，名次不靠前也欢迎讲。提前写明是想让你据此安排工作方式：只有你自己讲得出来的实现，才拿得到这 10 分。

6. 提交要求

请提交一个压缩包，命名为 学号_姓名.zip（例如 09023307_张三.zip），包含以下内容：

1. 源代码（/src）

完整可编译的 C++ 工程。单文件，或提供 Linux 下可直接 **make** 的 Makefile。禁止绝对路径。编译不通过 = 本轮无成绩。

2. 校验数据（fingerprint.txt）

对你通关的每一个 k ，记录 n 的取值与 3.1 节的 5 行输出，另附该档的用时与峰值内存。

3. 使用说明（README.md）

- 编译命令与运行命令示例；如果需要任何手工步骤，详细说明。
- 你的自测数据：每个 k 的用时与峰值内存，以及你是怎么测的。
- 你如何验证结果是对的。这一项是本题的重点而不是附属品：当结果有 7.6 亿位、无法逐位核实时，你凭什么相信它？